

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
1 ОБЩАЯ ПОСТАНОВКА ЗАДАЧИ NMT	5
1.1 Вероятностная постановка задачи	5
1.2 Токенизация текста	6
1.3 Метрики качества перевода	7
1.4 Методы регуляризации моделей NMT	9
2 АРХИТЕКТУРЫ МОДЕЛЕЙ NMT	12
2.1 Encoder-Decoder архитектура	12
2.2 Архитектура только декодера	13
2.3 Общая теория базовых блоков	13
2.4 LSTM Encoder-Decoder	16
2.5 Transformer	17
2.5.1 Transformer Encoder-Decoder	17
2.5.2 Transformer Decoder	17
2.6 Mamba	18
2.6.1 Mamba Encoder-Decoder гибрид с Attention	18
2.6.2 Mamba Decoder	18
3 ЭКСПЕРИМЕНТАЛЬНОЕ СРАВНЕНИЕ NMT ПОДХОДОВ	19
3.1 Подготовка данных	19
3.2 Среда экспериментов и гиперпараметры	21
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25

ВВЕДЕНИЕ

Нейронный машинный перевод (далее NMT) является одной из основных задач обработки естественного языка. С практической точки зрения он заключается в построении модели, которая по тексту на исходном языке формирует текст на целевом языке, сохраняя смысл, грамматическую корректность и контекст высказывания. В отличие от более ранних статистических подходов, нейронные модели позволяют обучать перевод как единую задачу преобразования последовательностей.

При построении системы NMT необходимо выбрать не один отдельный алгоритм, а целый набор связанных решений. К ним относятся архитектура модели, способ представления исходного и целевого текста, схема организации перевода, набор языковых направлений, подготовка обучающих данных и метод обучения. Каждый из этих выборов влияет на качество перевода, вычислительную сложность и возможность переноса модели на новые языки или домены. Поэтому такие решения важно рассматривать не изолированно, а в сравнении друг с другом.

С момента появления NMT было предложено несколько основных архитектурных подходов. К классическим решениям относятся рекуррентные модели LSTM с механизмом внимания [1]. Позднее основой большинства систем стали модели Transformer с механизмом самовнимания [2]. В последние годы также развиваются архитектуры, основанные на моделях пространства состояний, в частности Mamba [3]. Эти подходы по-разному решают задачу учета контекста, поэтому их сравнение позволяет понять, какие ограничения и преимущества возникают при обработке последовательностей разной длины и сложности.

Не менее важна схема организации самой модели перевода. В классическом варианте encoder-decoder encoder строит представление исходного предложения, а decoder по этому представлению формирует перевод. В decoder-only подходе исходный текст и целевой перевод рассматриваются как части одной авторегрессионной последовательности. Эти схемы могут использоваться с разными архитектурами, поэтому сравнение архитектур без учета способа организации модели дает неполную картину.

Отдельный уровень сравнения связан с языковым охватом. Модель может обучаться для одного направления перевода, например с одного фиксированного исходного языка на один целевой язык. Другой вариант — мультязычная модель, которая работает сразу с несколькими языками и направлениями перевода. Мультязычность позволяет использовать общие представления между языками, но одновременно усложняет балансировку данных и постановку обучения.

Качество NMT существенно зависит не только от модели, но и от обучающего корпуса. На практике применяют фильтрацию шумных параллельных предложений, удаление дубликатов, ограничения по длине и соотношению длин предложений, а также различные способы аугментации данных. Эти методы подготовки данных также требуют сравнения, поскольку улучшение корпуса иногда влияет на итоговое качество не меньше, чем замена архитектуры.

Различаются и подходы к обучению моделей перевода. Базовым является supervised learning, при котором модель обучается на парах исходных предложений и эталонных переводов. Дополнительно могут использоваться методы дообучения с учетом автоматических метрик, человеческих оценок или reinforcement learning. Таким образом, задача исследования сводится не только к описанию отдельных моделей, но и к сопоставлению архитектурных, языковых, корпусных и обучающих решений в общей постановке нейронного машинного перевода.

Постановка задачи

Целью работы является сравнительное исследование подходов к построению и обучению моделей NMT.

Для достижения поставленной цели необходимо решить следующие задачи:

- рассмотреть общую постановку задачи NMT;
- сравнить архитектуры LSTM, Transformer и Mamba в задачах перевода;
- сопоставить encoder-decoder и decoder-only схемы;
- охарактеризовать однонаправленные и мультязычные модели перевода;
- рассмотреть подходы к подготовке, фильтрации и аугментации

обучающих данных;

- описать основные подходы к обучению и оценке качества моделей перевода.

1 Общая постановка задачи NMT

1.1 Вероятностная постановка задачи

Машинный перевод заключается в построении текста на целевом языке по заданному тексту на исходном языке. В нейронном машинном переводе эта задача обычно формулируется как задача условного языкового моделирования. Перед подачей в модель текст преобразуется в последовательность токенов. В абстрактном виде токеном будем называть элемент конечного словаря: это может быть слово, часть слова, отдельный символ или специальный служебный элемент.

Пусть V_x — словарь исходного языка, а V_y — словарь целевого языка. Исходное предложение представим последовательностью токенов

$$x = (x_1, \dots, x_m), \quad x_i \in V_x,$$

а перевод — последовательностью токенов

$$y = (y_1, \dots, y_n), \quad y_t \in V_y.$$

Задача модели с параметрами θ состоит в оценке условной вероятности $p_\theta(y \mid x)$. Авторегрессионная декомпозиция этой вероятности имеет вид:

$$p_\theta(y \mid x) = \prod_{t=1}^n p_\theta(y_t \mid y_{<t}, x), \quad (1)$$

где в формуле (1) $y_{<t} = (y_1, \dots, y_{t-1})$ — уже построенная часть перевода. Таким образом, на каждом шаге модель предсказывает следующий токен перевода, используя исходное предложение и предыдущие токены целевой последовательности.

Для обучения используется параллельный корпус $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$, состоящий из пар исходных предложений и эталонных переводов. Параметры модели подбираются с помощью максимизации правдоподобия, что эквивалентно минимизации отрицательного логарифма вероятности

правильного перевода:

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \sum_{t=1}^{n_i} \log p_{\theta} \left(y_t^{(i)} \mid y_{<t}^{(i)}, x^{(i)} \right). \quad (2)$$

Функция потерь (2) задает штраф за низкую вероятность токенов, которые присутствуют в эталонном переводе. После обучения перевод для нового предложения можно записать как задачу поиска наиболее вероятной последовательности:

$$\hat{y} = \arg \max_y p_{\theta}(y \mid x).$$

На практике точный перебор всех вариантов невозможен, поэтому используют приближенные методы декодирования, например жадный поиск или beam search.

1.2 Токенизация текста

Нейронная модель не работает непосредственно со строкой текста. Перед обучением каждое предложение преобразуется в последовательность индексов словаря. Выбор способа токенизации влияет на длину последовательностей, размер словаря и способность модели обрабатывать редкие слова. По этой причине в машинном переводе часто используют подсловные токенизаторы, которые разбивают редкие слова на более частые фрагменты.

Одним из наиболее распространенных подходов является BPE (Byte Pair Encoding) [4]. Изначально BPE был предложен как метод сжатия данных, но в нейронном машинном переводе он используется для построения подслового словаря. Алгоритм начинает с разбиения слов на отдельные символы, после чего многократно объединяет наиболее частую пару соседних элементов. После заданного числа объединений получается словарь подсловных единиц. Такой подход позволяет представлять частотные слова одним токеном, а редкие слова — последовательностью более мелких фрагментов.

Другой подход называется WordPiece [5]. Он похож на BPE тем, что также строит словарь с помощью последовательного объединения более мелких единиц в более крупные. Однако критерий выбора объединения связан не только с частотой пары, но и с тем, насколько добавление нового токена улучшает вероятностную модель корпуса $\mathcal{C}$. В общем виде очередное объединение

можно описать как выбор пары, дающей наибольший прирост логарифма правдоподобия:

$$(a, b)^* = \arg \max_{(a, b)} \Delta \log P(\mathcal{C}).$$

На практике WordPiece часто помечает токены, продолжающие слово, специальным префиксом. Это позволяет отличать начало слова от его внутренней части.

Токенизатор Unigram использует другой принцип [6]. В нем заранее задается большой набор возможных подсловных единиц, а затем обучается вероятностная модель, в которой предложение рассматривается как результат выбора последовательности подсловных токенов. Если $u = (u_1, \dots, u_k)$ — один из возможных вариантов разбиения предложения на подслова, то его вероятность можно записать как

$$P(u) = \prod_{j=1}^k p(u_j).$$

Из всех допустимых разбиений выбирается наиболее вероятное:

$$u^* = \arg \max_{u \in S} P(u),$$

где S — множество возможных сегментаций исходной строки. В процессе обучения маловероятные подсловные единицы постепенно удаляются из словаря. Преимущество такого подхода состоит в том, что он естественно задает несколько возможных разбиений одного и того же текста, что может использоваться для регуляризации обучения.

1.3 Метрики качества перевода

Качество системы машинного перевода обычно оценивают сравнением с одним или несколькими эталонными переводами. Пусть $\hat{y}$ — перевод, построенный моделью, а r — эталонный перевод. Автоматическая метрика должна численно оценить, насколько $\hat{y}$ близок к r по форме или по смыслу.

Классической метрикой машинного перевода является BLEU [7]. Она основана на точности совпадения n -грамм между переводом модели и эталоном.

Для каждого порядка n вычисляется модифицированная точность p_n , в которой число совпавших n -грамм ограничивается их количеством в эталонном переводе. Итоговая оценка записывается как

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right),$$

где w_n — веса разных порядков n -грамм, а BP — штраф за слишком короткий перевод. Обычно используют $N = 4$ и равные веса. Метрика BLEU удобна для сравнения систем на корпусном уровне, но плохо учитывает синонимы и другие допустимые переформулировки.

Метрика chrF, или CHRF, оценивает совпадение символьных n -грамм [8]. В отличие от BLEU, она учитывает не только точность, но и полноту. Пусть P_{chr} — средняя точность по символьным n -граммам, а R_{chr} — средняя полнота. Тогда оценка имеет вид:

$$\text{chrF}_\beta = \frac{(1 + \beta^2)P_{chr}R_{chr}}{\beta^2 P_{chr} + R_{chr}}.$$

Так как сравнение выполняется на уровне символов, chrF лучше реагирует на частичные совпадения словоформ. Это особенно важно для языков с развитой морфологией, где разные формы одного слова могут иметь общие символьные фрагменты.

Более современным подходом является COMET [9]. В отличие от BLEU и chrF, эта метрика является обучаемой нейронной моделью. Она получает на вход исходное предложение, перевод модели и эталонный перевод, после чего предсказывает оценку качества:

$$q = f_\phi(x, \hat{y}, r),$$

где f_ϕ — обученная модель оценки, а q — предсказанная численная оценка. COMET обучается на данных с человеческими оценками переводов, поэтому может учитывать смысловую близость даже в тех случаях, когда поверхностное совпадение слов невелико. При этом такая метрика сложнее в использовании, так как требует отдельной предобученной модели и зависит от данных, на которых эта модель была обучена.

1.4 Методы регуляризации моделей NMT

При обучении моделей NMT число параметров обычно велико, а параллельный корпус ограничен по размеру и качеству. Поэтому модель может не только изучать общие закономерности перевода, но и запоминать частные особенности обучающих примеров. Для уменьшения такого эффекта используют методы регуляризации: они изменяют процесс обучения так, чтобы модель давала более устойчивые предсказания на новых данных.

Одним из распространенных методов является dropout [10]. Пусть $h = (h_1, \dots, h_d)$ — вектор активаций некоторого слоя, а p — вероятность зануления элемента. Во время обучения случайно выбирается маска

$$m_i \sim \text{Bernoulli}(1 - p), \quad i = 1, \dots, d.$$

После этого вместо исходного вектора h в следующий слой передается вектор

$$\tilde{h} = \frac{m \odot h}{1 - p}, \quad (3)$$

где $\odot$ обозначает покомпонентное умножение. Масштабирование в формуле (3) сохраняет математическое ожидание активаций: $\mathbb{E}\tilde{h}_i = h_i$. На каждой итерации обучения сеть фактически работает с разными подмоделями, из-за чего отдельные нейроны меньше зависят от фиксированного набора соседних признаков. В NMT dropout применяют внутри слоев модели: к embeddings, выходам attention и feed-forward блоков, а также к промежуточным представлениям между слоями.

Другой подход связан с ограничением величины весов модели. Если параметры модели обозначить через w , то при обычной максимизации правдоподобия выбираются веса, при которых велика вероятность обучающих данных $p(D | w)$. В байесовской постановке рассматривается другая величина — апостериорная плотность весов при условии данных:

$$p(w | D) = \frac{p(D | w)p(w)}{p(D)}. \quad (4)$$

Так как $p(D)$ не зависит от w , из формулы (4) получается MAP-оценка

$$\hat{w}_{MAP} = \arg \max_w (\log p(D | w) + \log p(w)). \quad (5)$$

Таким образом, в MAP-подходе по выражению (5) максимизируется апостериорная плотность весов $p(w | D)$, а не только правдоподобие данных $p(D | w)$. Если в качестве априорного распределения взять нормальное распределение $p(w) = \mathcal{N}(0, \sigma^2 I)$, то

$$\log p(w) = -\frac{1}{2\sigma^2} \|w\|_2^2 + C.$$

Отсюда получается функция потерь с ℓ_2 -регуляризацией [11]:

$$\mathcal{L}_{reg}(w) = \mathcal{L}(w) + \frac{\lambda}{2} \|w\|_2^2. \quad (6)$$

В формуле (6) большие веса штрафуются квадратичной добавкой и заранее считаются менее предпочтительными. В градиентном спуске это приводит к обновлению

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta \nabla_w \mathcal{L}(w_t), \quad (7)$$

где η — шаг обучения. Первый множитель в формуле (7) постепенно уменьшает веса, поэтому в оптимизаторах этот прием называют weight decay. В современных адаптивных оптимизаторах weight decay часто реализуют как отдельный шаг уменьшения весов, а не как часть градиента функции потерь [12].

Еще одним способом регуляризации является label smoothing [13]. В стандартном обучении правильный токен кодируется one-hot распределением: вся вероятность относится к одному элементу словаря. Для NMT такая постановка бывает слишком жесткой, поскольку один и тот же фрагмент текста может иметь несколько допустимых переводов. Пусть $K = |V_y|$ — размер целевого словаря, а y_t — правильный токен на шаге t . При label smoothing целевое распределение заменяется сглаженным распределением

$$\tilde{q}_t(k) = (1 - \varepsilon) \mathbf{1}\{k = y_t\} + \frac{\varepsilon}{K}, \quad (8)$$

где в формуле (8) ε — коэффициент сглаживания. Функция потерь тогда

записывается как

$$\mathcal{L}_{LS} = - \sum_t \sum_{k=1}^K \tilde{q}_t(k) \log p_\theta(k \mid y_{<t}, x).$$

Такой прием не позволяет модели становиться чрезмерно уверенной в единственном токене и обычно делает распределение выходных вероятностей более устойчивым.

2 Архитектуры моделей NMT

2.1 Encoder-Decoder архитектура

Архитектура encoder-decoder описывает перевод как две связанные стадии: кодирование исходного предложения и авторегрессионное порождение целевого предложения. В принятой постановке исходная последовательность имеет вид $x = (x_1, \dots, x_m)$, а целевая последовательность — $y = (y_1, \dots, y_n)$. Encoder преобразует исходные токены в набор скрытых представлений:

$$H = (h_1, \dots, h_m) = \text{Enc}_\phi(x_1, \dots, x_m),$$

где H можно рассматривать как память модели об исходном предложении. Decoder строит распределение следующего целевого токена, используя уже построенный префикс перевода и представления encoder:

$$\pi_t = \text{Dec}_\psi(y_{<t}, H), \quad p_\theta(y_t \mid y_{<t}, x) = \pi_t(y_t),$$

где $\theta = (\phi, \psi)$ — параметры всей модели. Тогда условная вероятность перевода записывается как

$$p_\theta(y \mid x) = \prod_{t=1}^n p_\theta(y_t \mid y_{<t}, x).$$

В такой формулировке encoder отвечает за представление исходного текста, а decoder — за построение перевода. Связь между ними может быть реализована по-разному: через последнее состояние encoder, через attention над всеми состояниями encoder или через более сложные механизмы доступа к памяти. Для NMT особенно важен вариант с attention, поскольку decoder получает возможность на каждом шаге выбирать релевантные части исходного предложения.

2.2 Архитектура только декодера

В decoder-only постановке отдельный encoder отсутствует. Исходное предложение и целевой перевод записываются в одну авторегрессионную последовательность. Например, можно задать префикс

$$q(x) = (\langle src \rangle, x_1, \dots, x_m, \langle tgt \rangle),$$

после которого модель должна продолжить последовательность токенами перевода. Условная вероятность в этом случае имеет вид

$$p_{\theta}(y \mid x) = \prod_{t=1}^n p_{\theta}(y_t \mid q(x), y_{<t}).$$

Такая схема использует каузальную маску: каждый токен может обращаться только к предыдущим токенам общей последовательности. Во время обучения функция потерь обычно считается только по целевым токенам, а исходные токены играют роль условия или промпта. Преимущество decoder-only подхода состоит в единой формулировке задачи генерации, а недостаток — в отсутствии отдельного представления исходного предложения и явного cross-attention между исходной и целевой последовательностями.

2.3 Общая теория базовых блоков

Архитектуры NMT различаются тем, какие блоки используются для обработки последовательности и передачи контекста между токенами. Для дальнейшего сравнения достаточно выделить три базовых механизма: LSTM, attention и Mamba.

LSTM является рекуррентной архитектурой, предназначенной для обработки последовательностей и уменьшения проблемы затухающих градиентов [14]. На каждом шаге слой получает входной вектор z_t , предыдущее скрытое состояние h_{t-1} и состояние памяти c_{t-1} . Обновление памяти

управляется входным, забывающим и выходным вентилями:

$$\begin{aligned}
i_t &= \sigma(W_i z_t + U_i h_{t-1} + b_i), \\
f_t &= \sigma(W_f z_t + U_f h_{t-1} + b_f), \\
o_t &= \sigma(W_o z_t + U_o h_{t-1} + b_o), \\
\tilde{c}_t &= \tanh(W_c z_t + U_c h_{t-1} + b_c), \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\
h_t &= o_t \odot \tanh(c_t).
\end{aligned} \tag{9}$$

Система уравнений (9) показывает, что состояние памяти позволяет LSTM переносить информацию через несколько шагов последовательности. Главное ограничение рекуррентного блока состоит в последовательной обработке токенов: шаг t зависит от состояния шага $t - 1$. Структура вычислений LSTM-ячейки показана на рисунке 1.

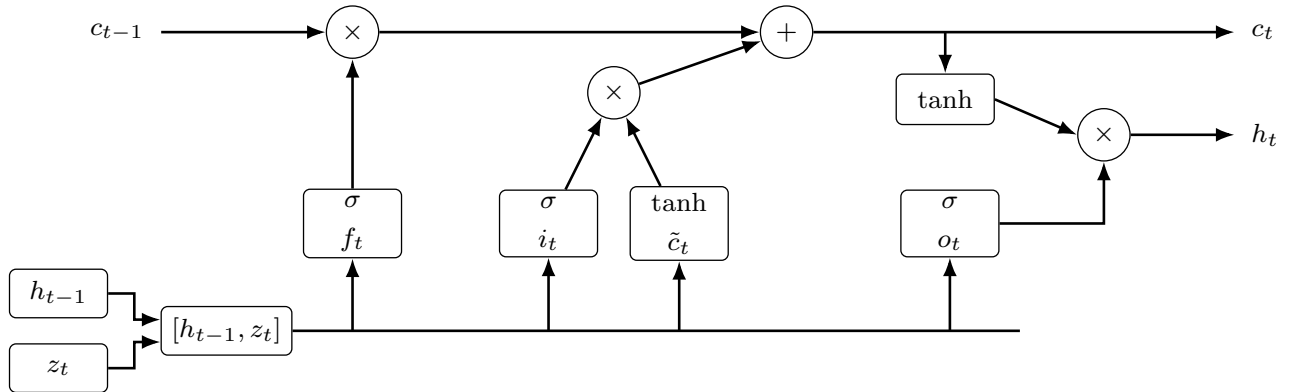


Рис. 1 – Блок-схема LSTM-ячейки

Attention позволяет модели напрямую обращаться к набору представлений, а не хранить весь контекст только в одном векторе [1, 2]. В общем виде attention вычисляет взвешенную сумму значений V по сходству запросов Q и ключей K :

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V, \tag{10}$$

где в формуле (10) d_k — размерность ключей. В self-attention запросы, ключи и значения строятся из одной и той же последовательности, поэтому токены внутри предложения обмениваются информацией друг с другом. В cross-attention запросы поступают из decoder, а ключи и значения — из en-

coder; так decoder получает доступ к представлениям исходного предложения. Последовательность операций scaled dot-product attention приведена на рисунке 2.

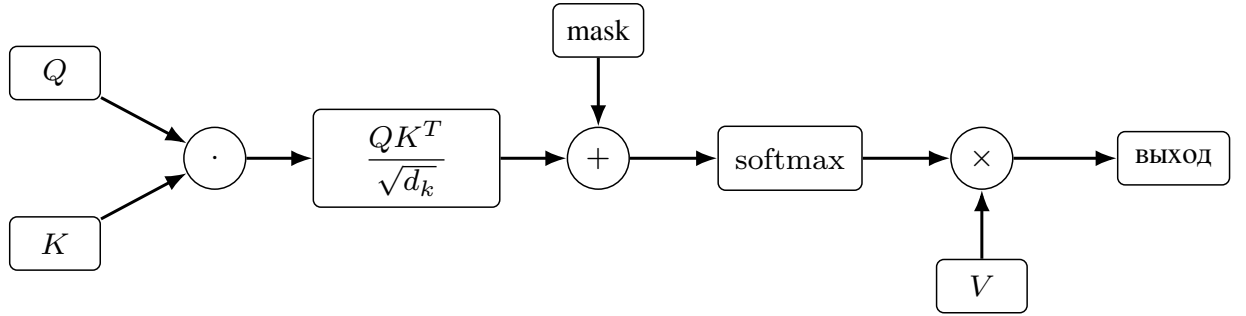


Рис. 2 – Блок-схема scaled dot-product attention

Multi-head attention применяет несколько attention-голов параллельно:

$$\text{МНА}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W^O, \quad (11)$$

В формуле (11) каждая голова имеет собственные проекции Q , K и V . Это позволяет модели одновременно учитывать разные типы связей между токенами. Структура multi-head attention представлена на рисунке 3.

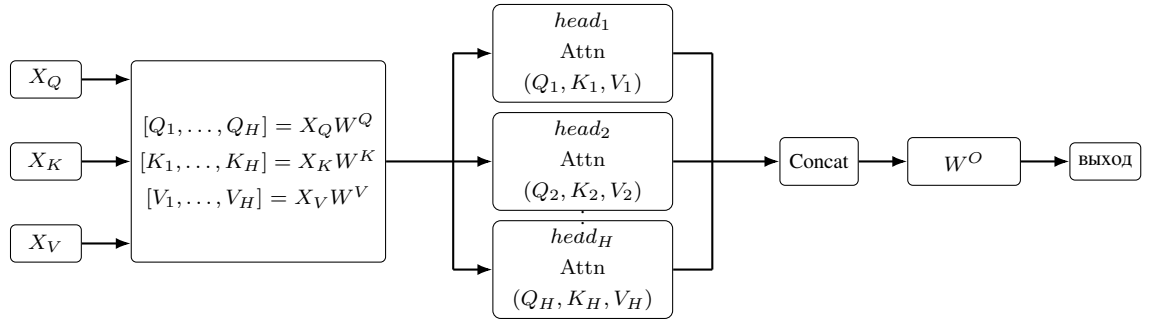


Рис. 3 – Блок-схема multi-head attention

В self-attention выполняется $X_Q = X_K = X_V$. В cross-attention последовательность запросов X_Q поступает из decoder, а X_K и X_V строятся по выходам encoder.

Mamba относится к моделям пространства состояний с избирательным обновлением состояния [3]. В упрощенном виде такой блок можно записать как

$$s_t = A_t s_{t-1} + B_t z_t, \quad r_t = C_t s_t, \quad (12)$$

где в формуле (12) s_t — скрытое состояние, z_t — входной вектор, а r_t —

выход блока. Ключевая особенность Mamba состоит в том, что параметры обновления зависят от текущего входа, поэтому модель может избирательно сохранять или забывать информацию. В отличие от self-attention, такой блок обрабатывает последовательность за линейное время по ее длине, что делает его привлекательным для длинных контекстов. Основные вычислительные ветви Mamba-блока показаны на рисунке 4.

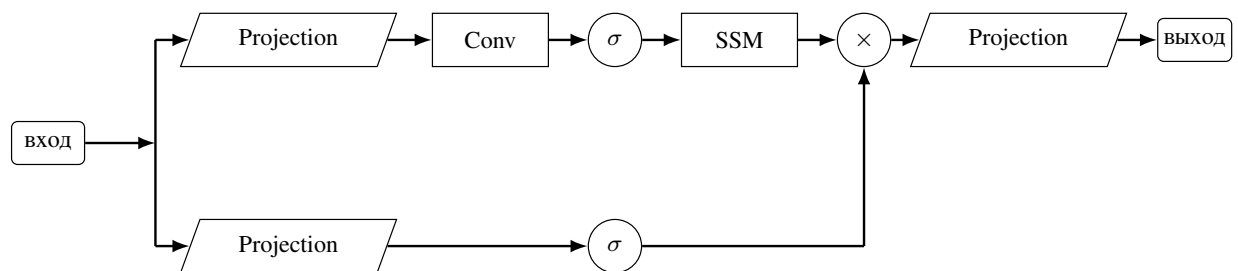


Рис. 4 – Блок-схема Mamba-блока

2.4 LSTM Encoder-Decoder

LSTM Encoder-Decoder является классической схемой нейронного машинного перевода [15, 1]. Encoder последовательно читает токены исходного предложения и строит скрытые состояния, содержащие информацию о прочитанном префиксе. Decoder также является LSTM и авторегрессионно генерирует перевод: на каждом шаге он получает предыдущий целевой токен и свое предыдущее состояние.

В раннем варианте вся информация об исходном предложении передавалась decoder через последнее состояние encoder. Для длинных предложений этого недостаточно, поэтому в NMT обычно используется attention над состояниями encoder. В этом случае decoder на каждом шаге формирует контекстный вектор как взвешенную сумму encoder-состояний и выбирает следующий токен с учетом этого контекста. Такая архитектура хорошо показывает основную идею encoder-decoder перевода, но хуже параллелизуется по сравнению с Transformer.

2.5 Transformer

2.5.1 *Transformer Encoder-Decoder*

Transformer Encoder-Decoder строится из стеков self-attention и полносвязных слоев [2]. Encoder обрабатывает исходное предложение с помощью self-attention, поэтому каждый токен может учитывать все остальные токены исходной последовательности. Decoder содержит causal self-attention, который запрещает смотреть на будущие токены перевода, и cross-attention, через который decoder обращается к выходам encoder.

Главное преимущество Transformer состоит в том, что self-attention позволяет параллельно обрабатывать все позиции последовательности во время обучения. Кроме того, cross-attention явно разделяет представление исходного текста и процесс генерации целевого текста. Ограничением является квадратичная сложность self-attention по длине последовательности, из-за чего обработка длинных текстов становится вычислительно дорогой.

2.5.2 *Transformer Decoder*

Transformer Decoder использует decoder-only постановку. Вместо отдельного encoder исходный текст и перевод записываются в одну последовательность, например в виде пары “исходное предложение — целевое предложение”. Модель обучается как авторегрессионная языковая модель: она видит исходные токены и уже построенную часть перевода, после чего предсказывает следующий целевой токен.

В такой архитектуре используется causal self-attention по всей объединенной последовательности. Преимущество подхода состоит в простоте: одна и та же модель может решать перевод и другие задачи генерации текста. Недостаток состоит в том, что нет отдельного cross-attention между encoder и decoder, поэтому связь исходного текста с переводом должна быть выучена внутри общего causal-контекста.

2.6 Mamba

2.6.1 Mamba Encoder-Decoder гибрид с Attention

Гибридная Mamba Encoder-Decoder архитектура сохраняет общую схему encoder-decoder, но заменяет self-attention-блоки на блоки Mamba. Encoder строит представления исходной последовательности с помощью Mamba-блоков; для encoder могут использоваться двунаправленные или некаузальные варианты, чтобы каждый токен учитывал контекст с обеих сторон. Decoder использует каузальные Mamba-блоки, так как перевод строится слева направо.

Связь между исходным предложением и переводом в такой схеме сохраняется через attention, обычно через cross-attention от decoder к выходам encoder. Иными словами, Mamba заменяет внутреннее self-attention-моделирование последовательностей, но decoder по-прежнему может обращаться к представлениям исходного предложения. Такой вариант является гибридом: он объединяет линейную обработку последовательности в Mamba с явным механизмом сопоставления исходных и целевых токенов через attention.

2.6.2 Mamba Decoder

Mamba Decoder является decoder-only моделью, в которой вместо causal self-attention используются каузальные Mamba-блоки. Как и в Transformer Decoder, исходное предложение и перевод представляются одной последовательностью, а функция потерь обычно считается по токенам целевого перевода. На каждом шаге Mamba обновляет скрытое состояние и на его основе предсказывает следующий токен.

Такой подход сохраняет простоту decoder-only формулировки и при этом имеет линейную сложность по длине последовательности. Это делает Mamba Decoder потенциально удобным для длинных входов и эффективного авторегрессионного декодирования. Однако отсутствие отдельного encoder и cross-attention означает, что вся информация об исходном предложении должна быть сохранена во внутреннем состоянии модели, поэтому качество перевода сильнее зависит от способности Mamba удерживать релевантный контекст.

3 Экспериментальное сравнение NMT подходов

3.1 Подготовка данных

В экспериментальной части рассматривается только русско-английская языковая пара. Все модели обучаются переводить предложения между русским и английским языками, поэтому подготовка корпуса направлена не на построение многоязычной системы, а на получение достаточно качественных параллельных пар для одного направления перевода. Для сравнения подходов используются три варианта набора данных разного размера.

Первый набор данных включает OPUS-100 [16] и Yandex English-Russian Parallel Corpus [17]. Он используется для подбора гиперпараметров, так как его размер позволяет проводить несколько серий обучения без чрезмерных вычислительных затрат. Второй набор данных дополнительно включает ParaCrawl [18]. Он применяется для анализа масштабируемости моделей при увеличении объема обучающих данных. Третий набор данных строится на основе OPUS-100, Yandex Corpus, ParaCrawl и United Nations Parallel Corpus [19]. Он используется для обучения и валидации лучшего из рассмотренных подходов.

Для всех трех вариантов корпуса выполняется одинаковая предварительная обработка. На первом этапе удаляются пары с сильным расхождением длины предложений. Пусть l_{ru} и l_{en} — длины русского и английского предложений после предварительной токенизации. Пара отбрасывается, если выполняется условие

$$\frac{|l_{ru} - l_{en}|}{\max(l_{ru}, l_{en})} > 0,3. \quad (13)$$

Фильтр (13) удаляет примеры, в которых одно предложение содержит только часть перевода другого предложения, лишний фрагмент текста или технический мусор. При этом порог не должен быть слишком жестким, поскольку естественная длина русского и английского переводов может различаться.

Следующим этапом является фильтрация по языку. Для этого используется идея language identification, то есть автоматического определения

языка текста. В общем виде такую задачу можно описать как выбор языка с максимальной апостериорной вероятностью:

$$\hat{c}(s) = \arg \max_{c \in C} p(c | s),$$

где s — входное предложение, C — множество возможных языков, а c — один из языковых классов. На практике такие модели часто используют символьные признаки и статистику коротких фрагментов текста, поскольку распределения символов и символьных n -грамм хорошо различают языки даже на относительно коротких предложениях [20]. Пара удаляется, если для русской стороны определяется не русский язык или для английской стороны определяется не английский язык.

После этого для каждой пары вычисляется reference-free версия COMET, то есть оценка качества без эталонного перевода. В этом случае модель получает только два предложения пары и оценивает, насколько одно из них является корректным переводом другого. Такую постановку обычно относят к *quality estimation*, поскольку качество оценивается без доступа к reference-предложению [21]. Если обозначить русское предложение через x , английское предложение через y , а модель оценки через f_ϕ , то вычисляемый score можно записать как

$$q = f_\phi(x, y).$$

После вычисления оценки остаются только пары, для которых выполняется условие $q > 0,7$. Этот этап позволяет удалить предложения, прошедшие простые фильтры длины и языка, но все еще являющиеся плохими переводными парами.

Дополнительно применяется аугментация с помощью LLM-модели. Она используется не для свободного перефразирования корпуса, а для исправления локальных ошибок в уже отобранных парах. К таким ошибкам относятся остатки разметки, технические символы, незначительные нарушения пунктуации, а также небольшие несоответствия по длине, когда одно предложение содержит начало или конец фрагмента, отсутствующий в другом предложении. Такой этап позволяет повысить однородность корпуса без изменения основной семантики переводной пары.

После всех этапов подготовки итоговые размеры наборов данных составляют:

1. OPUS-100 и Yandex Corpus — 1,9 млн пар предложений;
2. OPUS-100, Yandex Corpus и ParaCrawl — 4,7 млн пар предложений;
3. OPUS-100, Yandex Corpus, ParaCrawl и United Nations Parallel Corpus — 7,2 млн пар предложений.

3.2 Среда экспериментов и гиперпараметры

В качестве языка реализации используется Python, а модели строятся с помощью фреймворка `torch` [22]. Такой выбор удобен для экспериментальной работы с нейронными сетями, поскольку позволяет явно описывать архитектуры, контролировать цикл обучения и использовать аппаратное ускорение для матричных операций. Для корректного сравнения подходов используется единый обучающий цикл: различаются архитектуры моделей, но не базовая схема оптимизации и вычисления функции потерь.

Для оптимизации используется AdamW [12]. Данный алгоритм является модификацией Adam, в которой `weight decay` отделен от градиента основной функции потерь. Если g_t обозначает градиент на шаге t , то Adam оценивает экспоненциальные скользящие средние первого и второго моментов:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Параметры β_1 и β_2 задают скорость накопления этих средних. После коррекции смещения параметры модели обновляются с учетом `learning rate` η_t и отдельного коэффициента `weight decay` λ :

$$w_{t+1} = (1 - \eta_t \lambda) w_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}. \quad (14)$$

Обновление (14) удобно для NMT-моделей, поскольку регуляризация весов не смешивается с адаптивным масштабированием градиента.

В качестве основной функции потерь используется `cross entropy loss`. Для авторегрессионной модели перевода она имеет вид

$$\mathcal{L}_{CE} = - \sum_{t=1}^n \log p_{\theta}(y_t \mid y_{<t}, x), \quad (15)$$

где в формуле (15) x — исходное предложение, y_t — целевой токен на шаге t , а $y_{<t}$ — уже известный префикс перевода. При использовании label smoothing вместо one-hot распределения применяется сглаженное целевое распределение, что уменьшает чрезмерную уверенность модели в одном токене. Также в экспериментах используются dropout и weight decay. Таким образом, регуляризация включает три независимых механизма: зануление части промежуточных активаций, сглаживание целевых меток и ограничение величины весов через оптимизатор.

Learning rate изменяется по расписанию warmup + cosine annealing. На первых шагах обучения применяется warmup: learning rate постепенно увеличивается от малого значения до базового уровня. Это особенно важно для глубоких sequence-to-sequence моделей, поскольку в начале обучения оценки моментов в AdamW еще нестабильны, а слишком большой шаг может привести к резкому росту функции потерь. Идея warmup широко используется в обучении Transformer-моделей [2]. После завершения warmup learning rate уменьшается по косинусному закону [23]:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos \frac{\pi(t - T_w)}{T - T_w} \right), \quad t \geq T_w, \quad (16)$$

где в формуле (16) T_w — число warmup-steps, T — общее число шагов обучения, η_{max} — максимальное значение learning rate, а η_{min} — минимальное значение learning rate. Такое расписание используется потому, что оно сочетает осторожное начало обучения с плавным уменьшением шага на поздних этапах. В отличие от ступенчатого уменьшения learning rate, cosine annealing не создает резких скачков в динамике оптимизации.

Для ускорения обучения применяется automatic mixed precision. В качестве пониженной точности используется формат bfloat16, а не float16. Основная причина состоит в том, что bfloat16 сохраняет 8-битную экспоненту, как у float32, и поэтому имеет близкий к float32 диапазон представимых значений [24]. У float16 диапазон уже, поэтому при обучении глубоких моделей чаще возникают переполнения и исчезновение малых градиентов, а также может потребоваться дополнительное loss scaling. Формат bfloat16 имеет меньшую точность мантииссы, но для обучения нейронных сетей это обычно менее критично, чем устойчивый диапазон значений. Поэтому

использование `bfloat16` позволяет уменьшить потребление памяти и ускорить матричные операции без существенного изменения гиперпараметров обучения.

ЗАКЛЮЧЕНИЕ

В текущем состоянии работы рассмотрена общая постановка задачи NMT и описаны основные компоненты, необходимые для построения системы нейронного машинного перевода. Была сформулирована вероятностная постановка перевода как задачи условного языкового моделирования, рассмотрены способы токенизации текста, методы декодирования и метрики качества перевода. Также были описаны методы регуляризации моделей NMT, включая dropout, weight decay и label smoothing.

Отдельно рассмотрены архитектурные подходы, применяемые в NMT. В работе описаны encoder-decoder и decoder-only постановки, базовые рекуррентные модели, механизм attention, Transformer и архитектура Mamba. Для ключевых блоков приведены математические формулы и схемы, поясняющие принцип обработки последовательности.

В практической части начато описание экспериментального сравнения NMT-подходов. На данном этапе определены используемые русско-английские наборы данных, описана процедура предварительной фильтрации и аугментации корпуса, а также зафиксированы основные элементы экспериментальной среды и гиперпараметры обучения. В дальнейшем этот раздел должен быть дополнен описанием обучаемых моделей, результатами экспериментов и сравнительным анализом полученных метрик.

Таким образом, текущая версия работы содержит теоретическую основу и заготовку экспериментального протокола, необходимые для дальнейшего сравнения архитектур NMT на подготовленных корпусах данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Luong M.-T., Pham H., Manning C.D. Effective Approaches to Attention-based Neural Machine Translation [Электронный ресурс]. — 2015. — URL: <https://arxiv.org/abs/1508.04025>
2. Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A.N., Kaiser L., Polosukhin I. Attention Is All You Need [Электронный ресурс]. — 2017. — URL: <https://arxiv.org/abs/1706.03762>
3. Gu A., Dao T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces [Электронный ресурс]. — 2023. — URL: <https://arxiv.org/abs/2312.00752>
4. Sennrich R., Haddow B., Birch A. Neural Machine Translation of Rare Words with Subword Units [Электронный ресурс]. — 2016. — URL: <https://arxiv.org/abs/1508.07909>
5. Schuster M., Nakajima K. Japanese and Korean voice search [Электронный ресурс] // 2012 IEEE International Conference on Acoustics, Speech and Signal Processing. — 2012. — P. 5149–5152. — URL: <https://doi.org/10.1109/ICASSP.2012.6289079>
6. Kudo T. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates [Электронный ресурс]. — 2018. — URL: <https://arxiv.org/abs/1804.10959>
7. Papineni K., Roukos S., Ward T., Zhu W.-J. BLEU: a Method for Automatic Evaluation of Machine Translation [Электронный ресурс] // Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics. — 2002. — P. 311–318. — URL: <https://aclanthology.org/P02-1040/>
8. Popović M. chrF: character n-gram F-score for automatic MT evaluation [Электронный ресурс] // Proceedings of the Tenth Workshop on Statistical Machine Translation. — 2015. — P. 392–395. — URL: <https://aclanthology.org/W15-3049/>

9. Rei R., Stewart C., Farinha A.C., Lavie A. COMET: A Neural Framework for MT Evaluation [Электронный ресурс] // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. — 2020. — P. 2685–2702. — URL: <https://aclanthology.org/2020.emnlp-main.213/>
10. Srivastava N., Hinton G., Krizhevsky A., Sutskever I., Salakhutdinov R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting [Электронный ресурс] // Journal of Machine Learning Research. — 2014. — Vol. 15. — P. 1929–1958. — URL: <https://jmlr.org/papers/v15/srivastava14a.html>
11. Goodfellow I., Bengio Y., Courville A. Deep Learning [Электронный ресурс]. — 2016. — URL: <https://www.deeplearningbook.org/>
12. Loshchilov I., Hutter F. Decoupled Weight Decay Regularization [Электронный ресурс]. — 2017. — URL: <https://arxiv.org/abs/1711.05101>
13. Szegedy C., Vanhoucke V., Ioffe S., Shlens J., Wojna Z. Rethinking the Inception Architecture for Computer Vision [Электронный ресурс]. — 2015. — URL: <https://arxiv.org/abs/1512.00567>
14. Hochreiter S., Schmidhuber J. Long Short-Term Memory [Электронный ресурс] // Neural Computation. — 1997. — Vol. 9, no. 8. — P. 1735–1780. — URL: <https://doi.org/10.1162/neco.1997.9.8.1735>
15. Sutskever I., Vinyals O., Le Q.V. Sequence to Sequence Learning with Neural Networks [Электронный ресурс]. — 2014. — URL: <https://arxiv.org/abs/1409.3215>
16. Zhang B., Williams P., Titov I., Sennrich R. Improving Massively Multilingual Neural Machine Translation and Zero-Shot Translation [Электронный ресурс] // Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. — 2020. — P. 1628–1639. — URL: <https://aclanthology.org/2020.acl-main.148/>
17. Yandex LLC. License Agreement for the Use of the English-Russian Parallel Corpus [Электронный ресурс]. — URL: <https://yandex.ru/legal/corpus/en/?lang=en>

18. Esplà M., Forcada M., Ramírez-Sánchez G., Hoang H. ParaCrawl: Web-scale parallel corpora for the languages of the EU [Электронный ресурс] // Proceedings of Machine Translation Summit XVII: Translator, Project and User Tracks. — 2019. — P. 118–119. — URL: <https://aclanthology.org/W19-6721/>
19. Ziemski M., Junczys-Dowmunt M., Pouliquen B. The United Nations Parallel Corpus v1.0 [Электронный ресурс] // Proceedings of the Tenth International Conference on Language Resources and Evaluation. — 2016. — P. 3530–3534. — URL: <https://aclanthology.org/L16-1561/>
20. Lui M., Baldwin T. langid.py: An Off-the-shelf Language Identification Tool [Электронный ресурс] // Proceedings of the ACL 2012 System Demonstrations. — 2012. — P. 25–30. — URL: <https://aclanthology.org/P12-3005/>
21. Rei R., Treviso M., Guerreiro N.M., Zerva C., Farinha A.C., Maroti C., de Souza J.G.C., Glushkova T., Alves D.M., Lavie A., Coheur L., Martins A.F.T. CometKiw: IST-Unbabel 2022 Submission for the Quality Estimation Shared Task [Электронный ресурс]. — 2022. — URL: <https://arxiv.org/abs/2209.06243>
22. Paszke A., Gross S., Massa F., Lerer A., Bradbury J., Chanan G., Killeen T., Lin Z., Gimelshein N., Antiga L. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library [Электронный ресурс]. — 2019. — URL: <https://arxiv.org/abs/1912.01703>
23. Loshchilov I., Hutter F. SGDR: Stochastic Gradient Descent with Warm Restarts [Электронный ресурс]. — 2016. — URL: <https://arxiv.org/abs/1608.03983>
24. Kalamkar D., Mudigere D., Mellempudi N., Das D., Banerjee K., Avancha S., Vooturi D.T., Jammalamadaka N., Huang J., Yuen H. et al. A Study of BFLOAT16 for Deep Learning Training [Электронный ресурс]. — 2019. — URL: <https://arxiv.org/abs/1905.12322>